Flight Arrival Time Change Prediction

Project Overview

This project develops a machine learning system to forecast arrival delays for flights departing Dallas/Fort Worth International Airport (DFW). The project will aim to determine whether a flight will be delayed or not and the delay period by analyzing the airline operation data and the past weather conditions. Accurate delay prediction can improve airline scheduling, resource planning, and passenger communication.

The project deals with two prediction tasks. he first is a classification task, predicting whether a flight will be delayed or on time. The second is a regression problem, in which the model is used in estimating the number of delay minutes of delayed flights. By resolving both tasks, the system not only gives the delay status, but the anticipated severity of the delay.

Research Questions:

The below main questions are the focus of this study's investigation and resolution:

- Question 1 (Classification): Is it possible to forecast whether a flight will be late before it takes off?
- Question 2 (Regression): How many minutes late will an airplane arrive for flights that are expected to be delayed?

When combined, these inquiries create a useful end-to-end decision-support system. By classifying the high-risk flights with the help of the classifier, an operations team may apply the regression data to label the interventions according to the level of delay.

Data Filtering - Strategy & Results

The unfiltered ETA dataset contains multiple operational records for the same flight, as airlines issue updates throughout the flight lifecycle. Such updates contain estimated departures, after flight operation records, and repeated status updates. Consequently, some flights can be represented in the raw dataset numerous times. Since machine learning models require one representative row per flight, the raw operational records were filtered systematically to create a clean flight-level dataset.

The design of the filtering process was to maintain the most informative operational record of each flight and to eliminate the redundancies. Two main event types were used in this process: ETD (Estimated Time Departure) and CYCLE records. ETD records are in-flight operational updates, while CYCLE records are post-flight summaries. Priority was given to ETD records as they offer more pertinent in-flight indicators that can be used to forecast arrival delays.

Event Type	Description	When sent

ETD	Estimated Time of Departure update	Changes to the expected flight departure time (can happen before/after takeoff if there are delays or schedule changes).
POST	Multiple times during flight cycle	Updates that happen many times during the flight process (before, during, and after the flight).
CYCLE	Post-flight wrap record	Final report or record created after the flight is finished. Updates that happen once the plane has landed (like reaching gate or unloading).
Others	Various pre-departure signals	Any important updates before takeoff (like boarding, gate changes, or aircraft readiness).

Table1: Event Type: ETD vs CYCLE

Only records generated after the scheduled departure time were retained, ensuring that the flight was already airborne and that the ETA reflects actual in-flight conditions.

Because multiple ETD updates can exist for a single flight, a unique identifier was created using the airline code, flight number, and scheduled departure date. Based on this, only the earliest valid in-air ETD record (first post-departure update) was retained for each flight.

For flights without any valid ETD records, CYCLE records were used as a fallback. These were filtered using TTL_POST_QTY = 1, ensuring that only one final post-flight record was kept per flight. This guarantees that all flights are represented in the dataset.

After processing ETD and CYCLE records, both datasets were combined to form the final dataset. This step removed duplicate entries and ensured that each flight is represented by a single record.

Overall, this filtering process reduced the dataset size while preserving the most relevant information for delay prediction.

The stages of filtering and the number of rows obtained are summarized in the table below:

Stage	Row Count	Description
Raw ETA Records	2,739,387	All operational event records
ETD All Events	1,416,061	All ETD-type messages in the raw dataset
Post-Departure ETD	770,954	Only rows where post-departure MINS_TO_SCHD_DEP_QTY is less than 0 (in-air status).

ETD Unique Flights	461,839	Unique airline + flight + date combinations
Latest ETD Per Flight	288,806	One row per flight — earliest in-air estimate
CYCLE Fallback	834,003	Flights not covered by ETD (single-post CYCLE records)
Final ML Dataset	1,122,809	ETD rows + CYCLE fallback (no duplicates)

Table2: Data Pipeline Summary

The pipeline narrowed down the dataset of more than 2.7 million raw records to about 1.12 million distinct flights. The number of rows in every step was monitored closely: 1.4 million ETD events were narrowed to 461,839 unique flights, which was narrowed down to 288,806 most recent in-air posts. The CYCLE fallback has added 834,003 flights, making the final dataset to be 1,122,809 rows. This filtering was systematic and ensured that there was no duplication and that every flight was captured by one and consistent record.

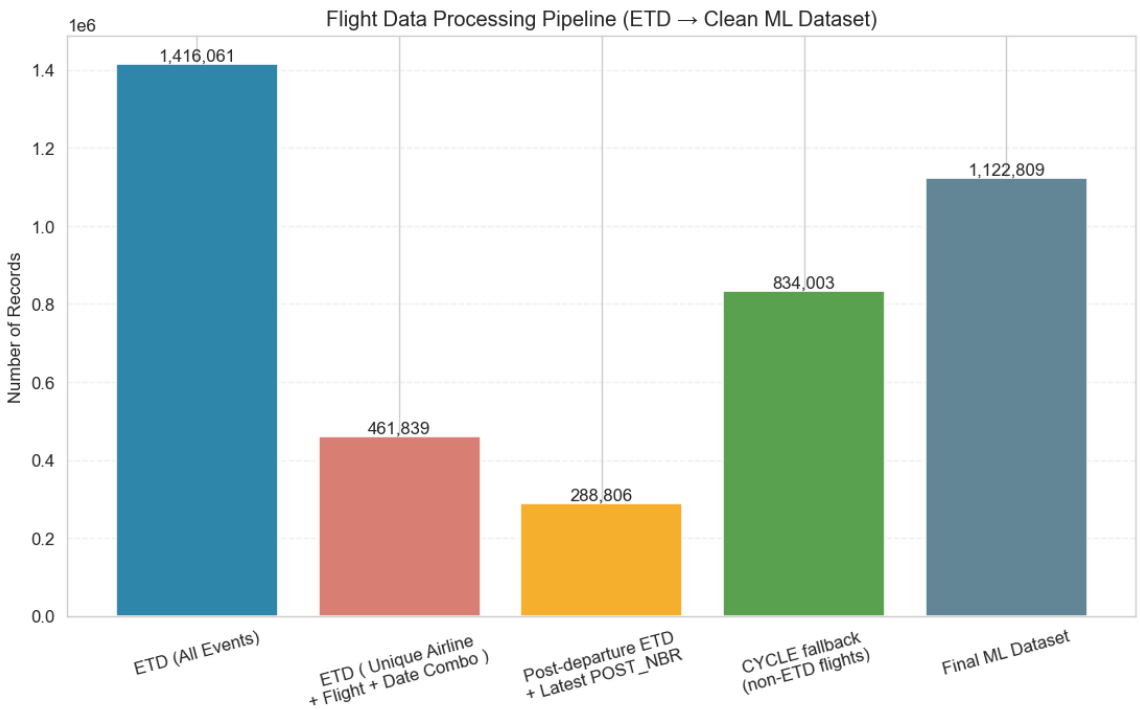

Fig1: ***Flight Data Processing Pipeline — ETD to Clean ML Dataset***

Target Variable

After we filtered the flight records, we then needed to select target variables to the machine learning models. Given that the project deals with classification and regression, the target variables were designed as one to identify a delayed flight and the other to quantify the delay minutes.

In the classification task, a binary target variable was formed, called is_delayed, to determine whether or not a flight was late. This variable was obtained based on arrival delay measure that was present in operational records. Flights whose arrival delay exceeded zero minutes were considered as delayed and those whose delay was zero or negative as on time. This transformation has transformed the delay prediction problem into a binary classification task in which the model determines whether to predict Delayed (1) or On-Time (0).

Class	Definition	Record Count	Percentage
0 — On Time / Early	Arrival variance <5 minutes	783,272	69.7%
1 — Delayed	Arrival variance ≥ 5 minutes	339,537	30.3%

Table3: Classification Target Distribution of Flight Records

The regression target is the actual delay in minutes. In case of flights arriving on time or early, the delay value was set to zero. The arrival variance was used in the case of delayed flights. Due to the very right-skewed nature of the distribution of delay minutes, a log transformation (log1p) was subsequently used to bring the values to a normal distribution and enhance the effectiveness of the models.

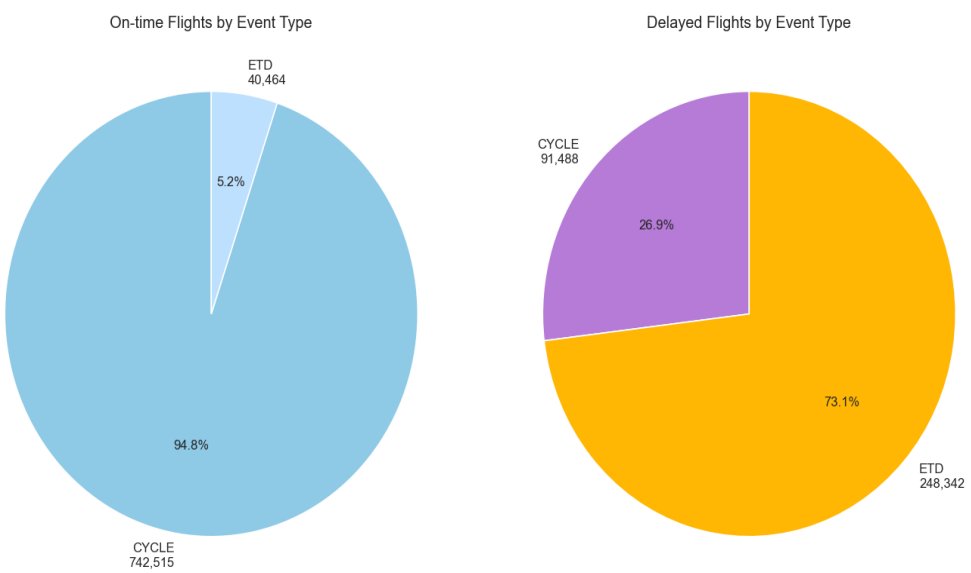

Fig2: On-Time vs. Delayed Flights by Event Type (ETD and CYCLE)

The target variables were determined following the completion of the ETD and CYCLE filtering phases to make sure that every flight had a single delay label. The is-delayed and delay-minutes were defined, and so the dataset was ready to complete two predictions is-delayed and delay-minutes. This two-fold goal method enhances the functionality of the system since it can not only identify operational delays but also forecast the severity. The pattern of the classification target distribution indicated that about 30.3 per cent of the flights were delayed whereas 69.7 per cent were punctual. This distribution shows that there is a moderate

level of imbalance in the classes, and this was taken into account in the model evaluation stage. Knowledge of the target distribution is fundamental because it helps to determine the evaluation metrics, as well as how to interpret classification performance.

Feature Engineering

Once the target variables have been defined, the second thing that followed was the formulation of meaningful predictor variables that could enhance the machine learning models performance. The raw operational data included flight event information, airport data, and weather information, but these variables needed to be transformed to be effectively utilized in predictive modeling. A feature engineering process was thus used to come up with operational, temporal and environmental features that reflect the key factors contributing to flight delays.

The initial set of engineered features was the time variables based on the planned departure timestamps. These were variables like hour of departure, day of the week, month and what was referred to as the weekend. These characteristics aid the models to capture recurrent patterns of delay related to certain period of the day, days of the week, and seasonal changes. As an illustration, any flight that takes off during the peak operational hours or during unfavorable season may tend to experience more delay.

Feature	Description
dep_hour	Hour of scheduled departure (0–23)
dep_dayofweek	Day of week (0 = Monday, 6 = Sunday)
dep_month	Month of year (1–12)
dep_year	Calendar year
is_weekend	Binary: 1 if Saturday or Sunday
is_holiday_season	Binary: 1 if November, December, or January
is_peak_hour	Binary: 1 if departure is 6–9 AM or 3–7 PM
dep_month_sin / dep_month_cos	Cyclical encoding of month to preserve circular nature
schd_dep_dt_SIN / schd_dep_dt_COS	Cyclical encoding of departure time-of-day in minutes
schd_expected_flight_duration	Scheduled flight duration in minutes (arrival – departure)

Table4: Temporal and Scheduling Features for Flight Delay Prediction

Weather Features for Flight Delay Prediction

To enhance the predictive power of the models, the dataset was supplemented with historical hourly weather data obtained through the Open-Mateo Archive API (for departure airport – Dallas Fort Worth (DFW)). The dataset included weather variables that included temperature, relative humidity, wind speed, wind gusts, precipitation, and surface pressure. The above features enable the models to take into consideration the conditions in the environment that could interfere with flight activities. As an example, high wind speeds, high rainfall, and storms tend to enhance the possibility of delays during departure and arrival.

Besides the raw weather measurements, there were derived weather indicators that were designed to reflect harsh operation conditions in a more efficient manner. These consisted of binary alerts of thunderstorms, intense rainfall, strong winds, and low visibility. By designing these indicators, the machine learning models can easily identify the occurrence of high-risk weather events, unlike when they use only raw numerical weather measurements.

The records of the flight events were also engineered into operational features so that to maintain important flight status information that could be used in delay prediction. These were variables about the estimated departure time, measure of operational updating, and the measure of arrival variance. These features capture real time operational indicators which are highly related to end arrival performance. The inclusion of these operational indicators enhances the detection of delays due to dynamic operational factors like change in departure timing and post-departure status update by the model.

Feature	Type	Count	Mean	Min	Max	Description
dfw_temperature_2m	Continuous	1,122,809	21.20	-17.5	43.3	Air temperature at 2m (°C)
dfw_relativehumidity_2m	Continuous	1,122,809	59.88	8.0	100.0	Relative humidity (%)
dfw_dewpoint_2m	Continuous	1,122,809	12.08	-27.0	27.4	Dew point temperature (°C)
dfw_apparent_temperature	Continuous	1,122,809	20.76	-22.7	45.7	Feels-like temperature (°C)
dfw_precipitation	Continuous	1,122,809	0.17	0.0	24.0	Total precipitation (mm)
dfw_rain	Continuous	1,122,809	0.17	0.0	24.0	Rainfall component (mm)
dfw_weathercode	Ordinal	1,122,809	8.26	0.0	75.0	Weather condition code

dfw_pressure_msl	Continuous	1,122,809	1016.11	992.8	1044.9	Mean sea level pressure (hPa)
dfw_surface_pressure	Continuous	1,122,809	994.88	972.2	1020.7	Surface pressure (hPa)
dfw_cloudcover	Continuous	1,122,809	45.99	0.0	100.0	Total cloud cover (%)
dfw_cloudcover_low	Continuous	1,122,809	18.08	0.0	100.0	Low cloud cover (%)
dfw_windspeed_10m	Continuous	1,122,809	13.92	0.0	48.0	Wind speed (km/h)
dfw_windgusts_10m	Continuous	1,122,809	27.37	1.1	86.4	Wind gusts (km/h)
dfw_winddirection_10m	Continuous	1,122,809	169.87	0.0	360.0	Wind direction (degrees)
dfw_shortwave_radiation	Continuous	1,122,809	306.61	0.0	1000.0	Solar radiation (W/m^2)
dfw_is_day	Binary	1,122,809	0.78	0	1	Daytime indicator
dfw_sunshine_duration	Continuous	1,122,809	2198.40	0.0	3600.0	Sunshine duration (sec)
dfw_wx_severity	Ordinal	1,122,809	0.50	0	3	Weather severity index
dfw_is_precip	Binary	1,122,809	0.13	0	1	Precipitation occurrence
dfw_is_thunder	Binary	1,122,809	0.00	0	0	Thunderstorm indicator
dfw_high_wind	Binary	1,122,809	0.15	0	1	Wind ≥ 40 km/h
dfw_severe_wind	Binary	1,122,809	0.01	0	1	Wind ≥ 60 km/h
dfw_heat_stress	Binary	1,122,809	0.00	0	1	High temp + humidity
dfw_fog_risk	Binary	1,122,809	0.09	0	1	Fog/low visibility risk
dfw_heavy_precip	Binary	1,122,809	0.01	0	1	Heavy rainfall indicator

Table 5: DFW Weather — Binary/Flag Feature Distributions

Continuous Weather Feature Distributions

The continuous weather variables were plotted using distribution plots to gain a better insight into the environmental conditions of the dataset. Such variables were temperature, relative humidity, wind speed, wind gust, precipitation, and surface pressure. Compared to the distributions, analyzing the distributions will assist in determining the usual weather conditions when departing and extreme weather values that can influence the performance of the flight.

This analysis revealed that most flights were conducted in the usual weather conditions, but some of the variables, especially precipitation and wind gust, had some extreme values irregularly. These circumstances can be associated with the disruptive weather conditions that enhance the chances of delays.

The distribution patterns in the Figure3 indicate that although the vast majority of weather patterns occur in a steady state, there are also extreme weather conditions that can be observed in a part of the flights and can significantly contribute to delay prediction.

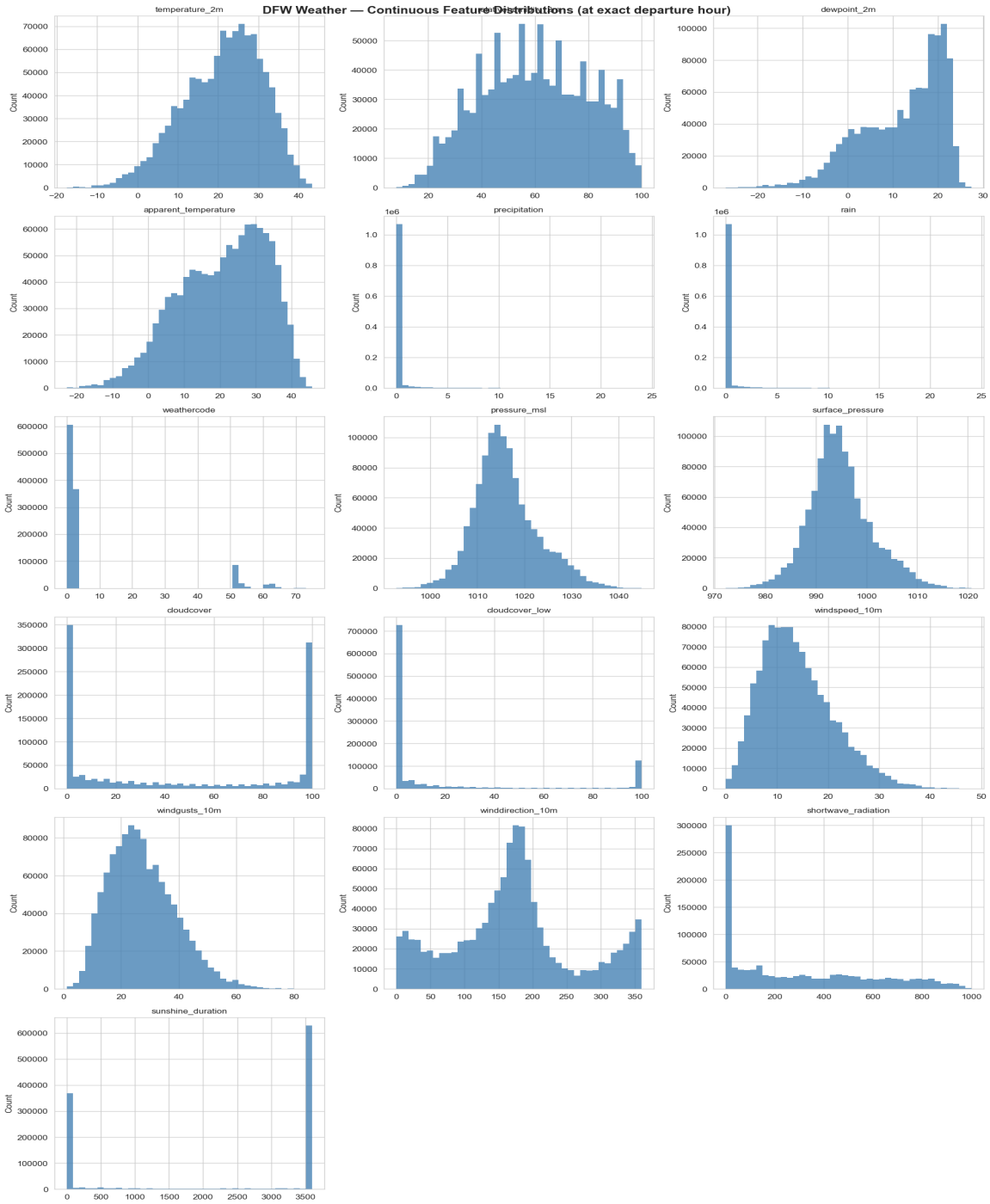

Fig3: Weather — Continuous Feature Distributions at Departure Hour

Weather Correlation with Delay Targets

The correlation analysis between the weather variables and the target variables is_delayed and delay_minutes were done to test the relationship between the weather conditions and the flight delays. This analysis can be used to determine the weather conditions that are most closely related to flight delays.

The findings revealed that the weather conditions, including the wind speed, wind gust and precipitation, had positive relations with the delay targets, which indicates that poor weather conditions enhance the probability and time of delays. In the same manner, binary weather predictors like storm and heavy rain flags were found to be significantly related to late flights

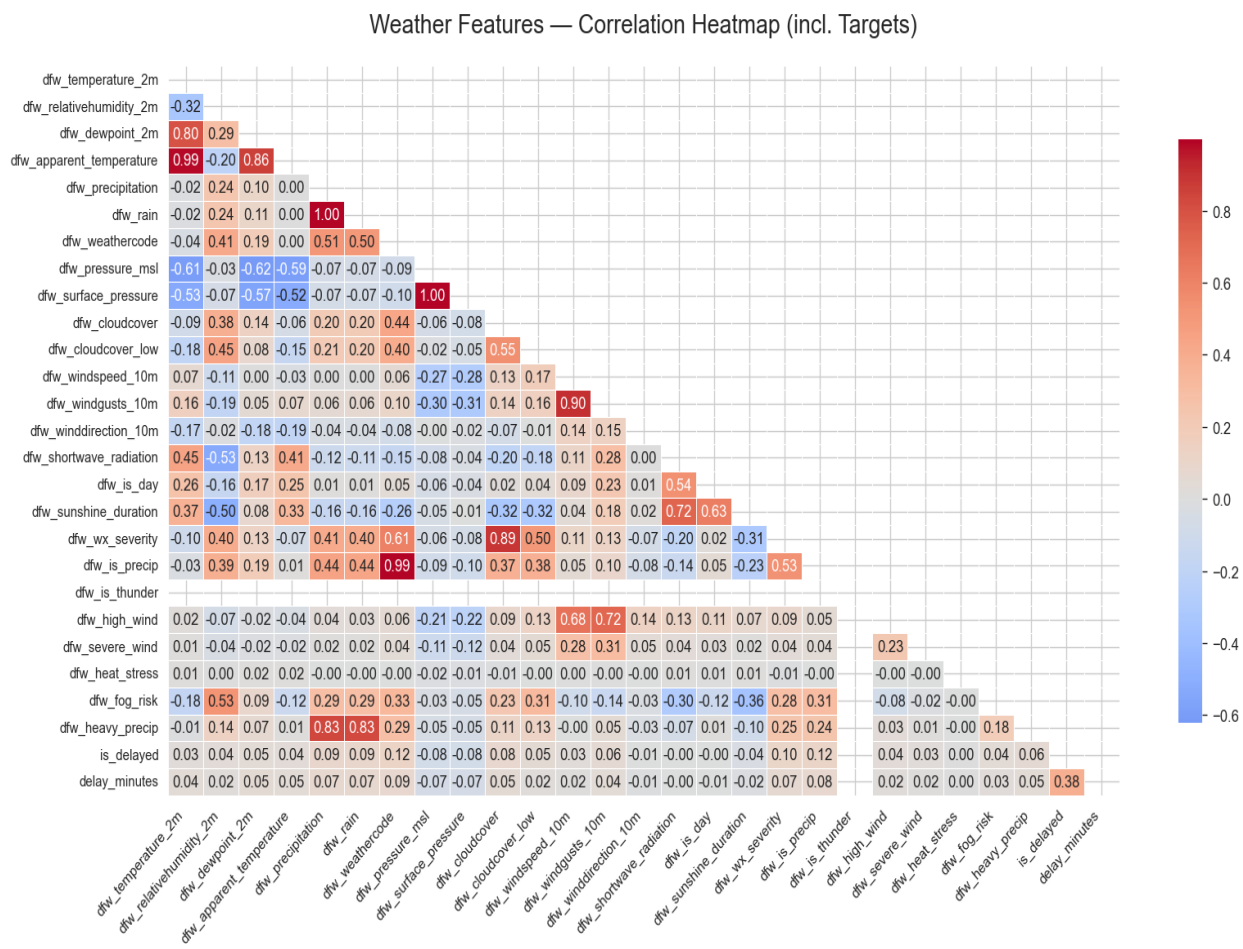

Fig4: Weather Features – Correlation Heatmap

These findings confirm that weather-related factors are important predictors and support their inclusion in the machine learning models.

Data Preprocessing

The dataset was thoroughly cleaned and converted before creating the predictive models to ensure consistency and reliability. The initial process was to clean the data, which consisted of deleting duplicates with the help of the composite flight identifier constructed by using airline code, flight number, and scheduled departure date. Incomplete values were treated systematically: flights where there were no valid ETD updates were filled with CYCLE records and missing values of arrival variance were imputed as zero

to indicate on time arrivals. Records that were invalid like those that had negative scheduled time or inconsistent time stamps were not considered to ensure integrity of the data set. To deal with outliers, boxplots have been created on continuous variables (delay minutes, wind speed and precipitation). These plots showed that there was right skewness on the delay minutes with few numbers of flights having extreme delays. A log transformation (log1p) was used to delay minutes to minimize the effect of these outliers. In the case of weather variables, binary flags were employed to represent severe weather conditions such as heavy rain, high wind, thunderstorms and low visibility such that extreme values were both represented and did not distort the numeric distributions.

Feature Selection

In machine learning, feature selection is a significant stage in the pipeline as it eliminates irrelevant or redundant variables, lessens noise, enhances computational efficiency, and aids in preventing data leakage. The models can also concentrate on the variables that have meaningful information to predict flight delays, by including just the relevant predictors.

Multicollinearity

Multicollinearity arises when two or more predictor variables are highly correlated and this may result in unstable estimates of coefficient, lower interpretability and adversely impact the performance of models, e.g. logistic regression. To evaluate this, pairwise correlations of the numerical features were analyzed with the help of a correlation matrix. Variables with high levels of correlation were checked to identify whether they were redundant data. Where there were duplicated features that had similar operational or weather patterns, redundant variables were eliminated to enhance a more stable model and eliminate the redundancy in the feature space. This measure made sure that the resulting feature set was informative as well as reduced redundancy which resulted in a stronger and interpretable predictive model. Features with a correlation greater than 0.80 were considered highly collinear, and one variable from each such pair was removed to reduce redundancy and improve model stability.

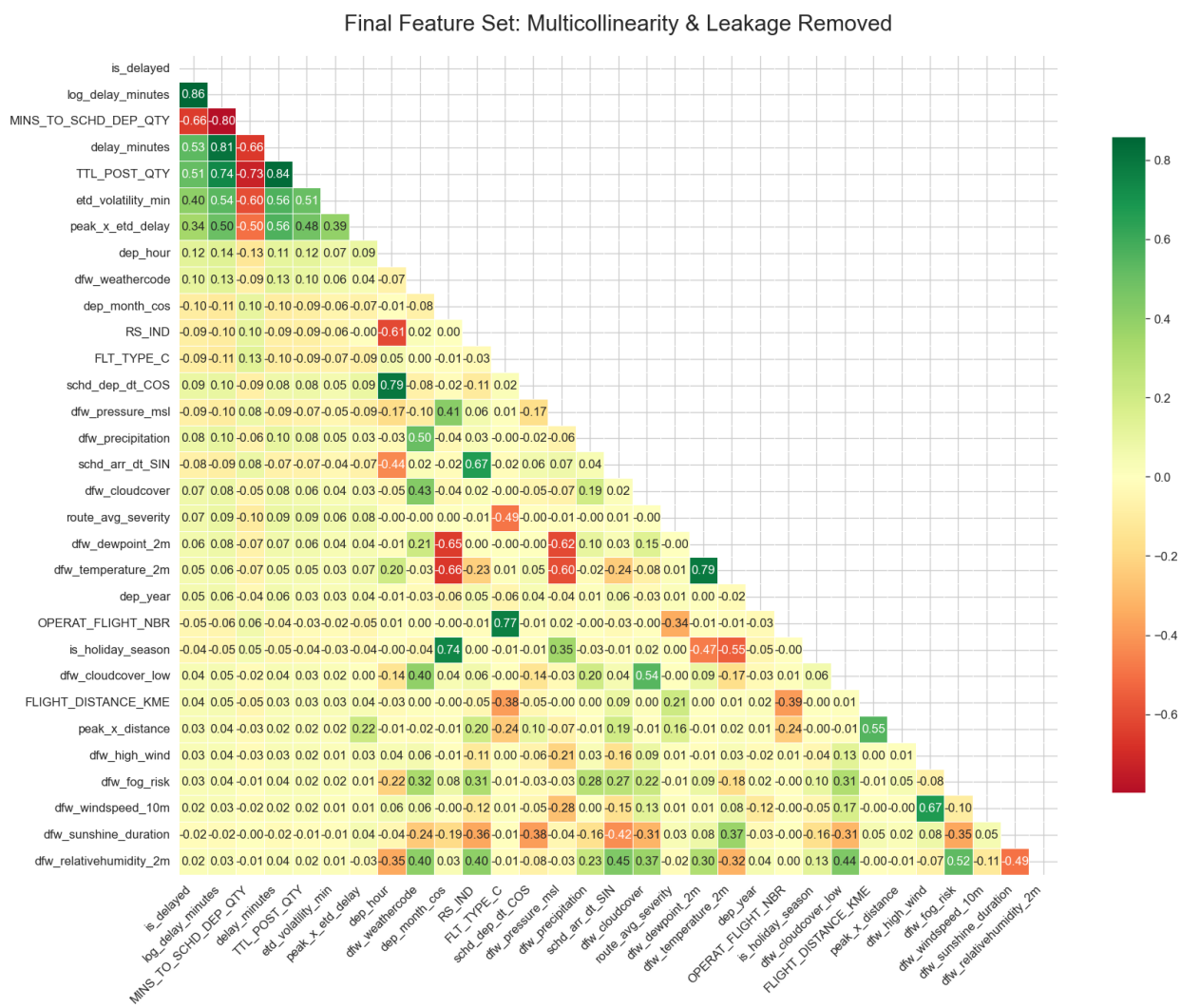

Fig5: Final Feature Set: Multicollinearity & Leakage Removed

Figure5 shows the pairwise correlations between the chosen features with the leakage variables and redundant features eliminated. The visualization proves that the final set of features has moderate correlations with no over-overlaps, which proves a balanced and strong modeling pipeline.

METHODOLOGY

Temporal Train/Test/Validation Split

The temporal split was performed based on the scheduled departure timestamp (Scheduled Departure Date/Time - It is the planned date and time when a flight is supposed to depart (as per the schedule), before any delays or changes.)), which contains both date and time, ensuring that flights are ordered chronologically before splitting. Flight delay prediction is inherently a time-series forecasting problem. In

production, a model trained on historical flights must predict delays for future flights it has never seen. This means the evaluation methodology must mimic that reality.

Random splitting violates this principle because:

• It allows the model to "see" future flights during training (e.g., train on a flight from March and test on one from January of the same year).

• It inflates performance metrics — the model appears to perform better than it would in deployment.

• Temporal patterns like seasonality, year-over-year trends, and gradual operational changes are not properly tested.

• Route delay probabilities and airline history features (computed as expanding means) would become leaky under random split.

With temporal split:

• Training data covers the earliest 70% of the timeline (January 2021 – approximately mid-2024).

• Test data covers the next 20% (approximately mid-2024 – early 2025).

• Validation data covers the final 10% (latest months, up to July 2025).

Split	Row Count	Share	Time Period (Approximate)
Training	767,326	70%	Jan 2021 – Mid 2024
Test	219,236	20%	Mid 2024 – Early 2025
Validation (Held Out)	109,619	10%	Early 2025 – Jul 2025
Total	1,096,181	100%	Jan 2021 – Jul 2025

Table6: Dataset Split Summary for Model Development

Classification Model:

The task of classification is to determine the delay or on-time flight basing on the operational and environmental characteristics. To do this, several machine learning models were deployed and tested with each other to determine the most effective model. The models that will be used are both linear and ensemble-based algorithms to model various kinds of relationships in the data.

Classification models used:

Logistic Regressionwas used as the baseline model because it can give an interpretable linear decision boundary and is a handy benchmark to assess more complicated algorithms.

Random Forest was used as an ensemble model that combines multiple decision trees to improve predictive performance and capture non-linear relationships between the input features and flight delays.

XGBoost was chosen since it is a strong gradient boosting algorithm that can learn the multifaceted interaction between features effectively and can handle class imbalance.

LightGBM was also an optimized gradient boosting algorithm that is efficient to train large datasets and high predictive performance.

Table7 below presents the model parameters

Model	Parameters
XGBoost	n_estimators = 300, max_depth = 6, learning_rate = 0.05, subsample = 0.8, colsample_bytree = 0.8, scale_pos_weight = (class imbalance ratio), eval_metric = "logloss", tree_method = "hist", random_state = 42
LightGBM	n_estimators = 300, max_depth = 6, learning_rate = 0.05, subsample = 0.8, colsample_bytree = 0.8, class_weight = "balanced", random_state = 42
Random Forest	n_estimators = 100, max_depth = 12, class_weight = "balanced", max_samples = 0.1, random_state = 42
Logistic Regression	max_iter = 1000, class_weight = "balanced", random_state = 42

Table7: Classification models and their parameters

Classification Results –Testset

Four classification models were trained to predict is_delayed (binary: delayed vs. on time). On the held-out 20% test set, all four models were assessed using AUC-ROC, Accuracy, Recall for the delayed class, and F1-Score for the delayed class**. Table 8** below displays the results.

Model	Accuracy	Recall (Delayed)	F1 Score (Delayed)	AUC-ROC
XGBoost	0.8436	0.6707	0.7724	0.8629
LightGBM	0.8434	0.6708	0.7723	0.8627
Random Forest	0.8444	0.6597	0.7705	0.8606
Logistic Regression	0.8443	0.6594	0.7702	0.8492

Table8: Classification Model Comparison – Test Set Results

The results show that XGBoost achieved the highest AUC-ROC score of 0.8629, indicating the best overall ability to distinguish between delayed and non-delayed flights. LightGBM produced nearly identical

performance, while Random ForestandLogistic Regression achieved slightly higher accuracy but lower delayed-flight recall and lower AUC-ROC values.

Although Random Forest and Logistic Regression showed marginally better overall accuracy, XGBoost provided better balanced performance for identifying delayed flights**,** which is particularly important in flight delay prediction. For this reason, XGBoost was identified as the strongest classification model among the four algorithms tested.

ROC Curve Comparison

To compare the performance of different classifiers, ROC curves were generated using the held-out test set. The curves illustrate the trade-off between True Positive Rate (TPR) and False Positive Rate (FPR).

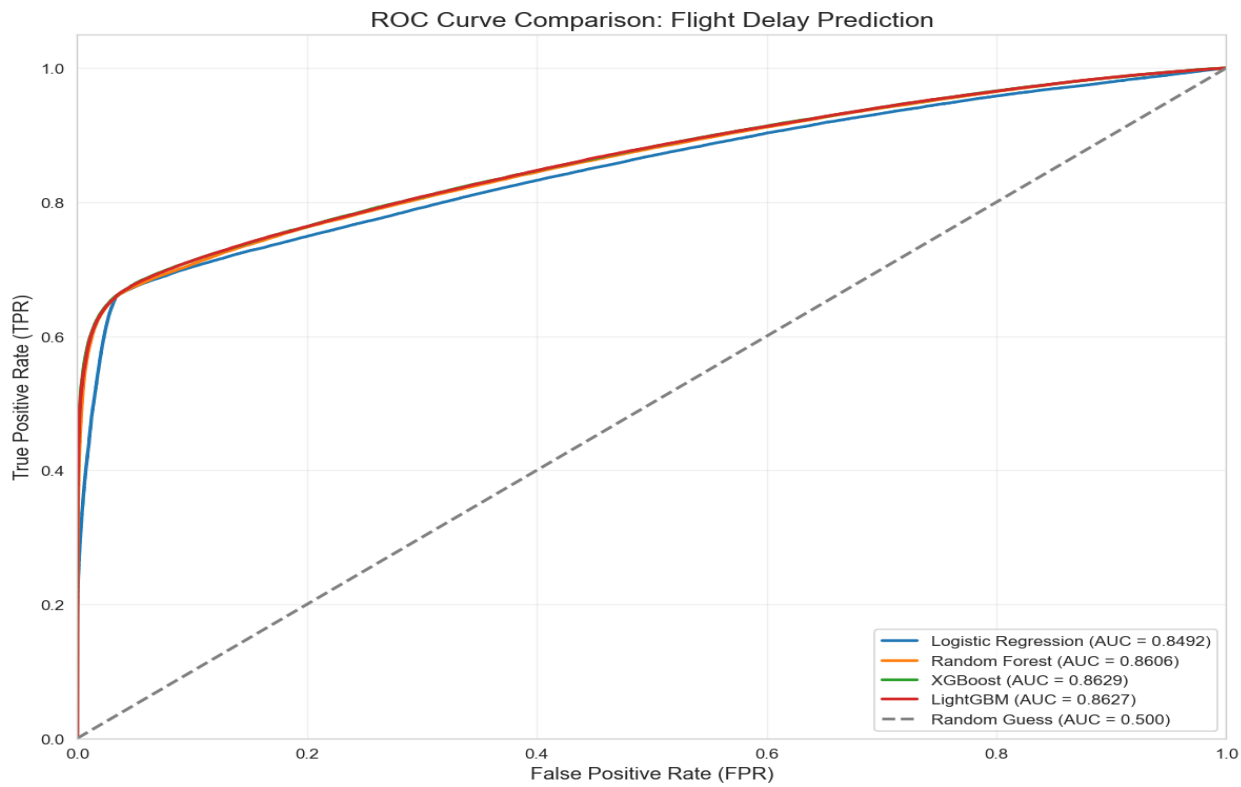

Fig6: ROC Curve Comparison – All Classification Models

Figure6 shows the ROC curves for the Logistic Regression (AUC = 0.8492) model, Random Forest with (AUC = 0.8606), XGBoost with (AUC = 0.8629), and LightGBM with (AUC = 0.8627). The random guess baseline with (AUC = 0.500) is included for reference. The results confirm that ensemble methods, particularly XGBoost and LightGBM, achieved the highest predictive performance.

CLASSIFICATION VALIDATION RESULTS (HELD OUT 10%)

Following a comparison of the four classification algorithms on the testing dataset, the XGBoost was found to be the best-performing model, in terms of the highest AUC-ROC score. In order to verify generalization to unseen data, the model was also tested on the held-out validation dataset, which was the last 10 percent of the records.

Validation dataset was not trained or compared to another model, and it is a separate benchmark to test the robustness of the model. The validation data was used to apply the XGBoost classifier whose performance was assessed through Accuracy, Precision, Recall, F1-score and AUC-ROC.

Metric	Value
Validation Accuracy	0.8257
Validation AUC-ROC	0.8622
Precision (Delayed)	0.92
Recall (Delayed)	0.68
F1-Score (Delayed)	0.78

Table9: Classification Model Comparison – Test Set Results

The validation findings indicate that the XGBoost classifier has AUC-ROC score of 0.8622 and accuracy of 82.57 indicating that the algorithm is a strong predictive of the unknown validation data.

This model had a precision of 92% in terms of delayed flights which means that in most cases, the prediction of the model was accurate when it was predicted to be delayed. This sensitivity is useful in the workplace where we would want to reduce the number of false alarms.

The delayed flight recall rate was 68% which indicates that the model was able to identify most of the delayed flights, but not all of the delayed flights were captured. The F1-score of 0.78 represents an equal trade-off between precision and recall.

These validation findings indicate that the XGBoost model was still able to maintain predictive accuracy outside the testing data, which means that the model could generalize well, and thus, it can be used to predict the operational delay.

CLASSIFICATION CONFUSION MATRIX

To assess the classification performance of the chosen XGBoost model, a confusion matrix was considered on the held-out validation dataset. The confusion matrix is used to give a detailed analysis of the correct and incorrect predictions of both delayed and on-time flights. Based on the confusion matrix, the model was generally very predictive as it predicted 56,400 on-time flights and 34,114 delayed flights correctly, which demonstrates its predictive power in both classes.

The false positive cases were 3,118 which were the number of true on-time flights that were forecasted as delayed. These are those flights that had been punctual but were wrongly rated as being delayed. The false negative cases were the number of true delayed flights that were predicted to be on-time 15,987.

They are late flights which the model could not detect. According to the confusion matrix, the model is especially good at recognizing on-time flights since the number of records that are correctly identified as on-time and the number of false positives is significantly high and low respectively.

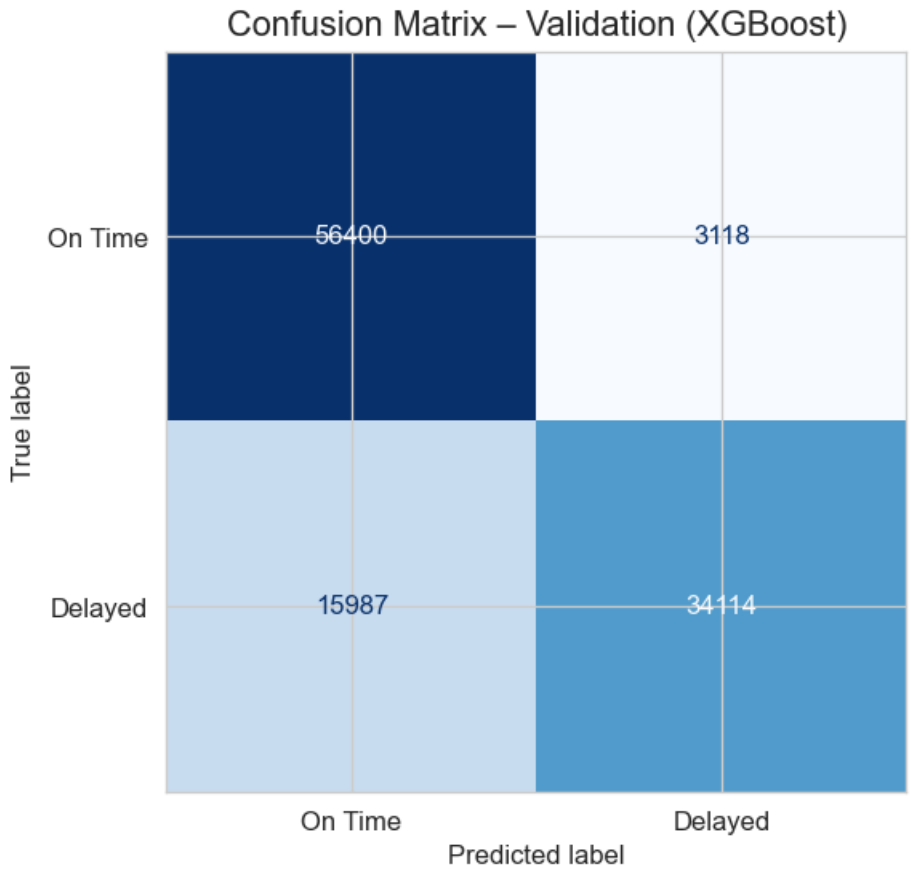

Fig7: Confusion Matrix – XGBoost Classification Validation (Held-Out 10%)

Regression Model:

The regression dataset comprised of all the flights, both on-time and delayed flights, but the delay duration was entered as zero. This made sure that the models got to learn about all the outcomes of the flights that included delayed and non-delayed flights. The ready dataset was chronologically divided into training, testing and validation subsets with the same 70, 20, 10 percent split strategy as in the classification task. This translated to 767,326 training records, 219,236 test records and 109,619 validation records. Leakage check was also done to ensure that there were no target related variables in the predictor set and the models were only trained on valid explanatory variables.

Two gradient boosting regression models were trained for predicting delay duration: XGBoost Regressor and LightGBM Regressor. The choice of these models is based on the fact that gradient boosting methods are very effective when working with structured tabular data and can be used to model non-linear relationships between operational variables, weather features, and delay duration. Both models were set up with 500 estimators, a maximum tree depth of 6, and a learning rate of 0.05, which offers enough complexity to the model, and still allows it to generalize.

REGRESSION RESULTS – TEST SET

The testing data were tested on the models in terms of Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 Score. These metrics were selected to evaluate the accuracy of prediction in various aspects. MAE is the mean absolute discrepancy between the predicted and actual delay minutes, RMSE highlights greater prediction errors and the R 2 Score indicates the extent to which the model accounts for the variation in delay duration.

Model	MAE (Minutes)	RMSE (Minutes)	R^2 Score
XGBoost Reg	7.17	17.26	0.7693
LightGBM Reg	19.64	51.69	0.7028

Table10: Regression Results – Test Set

The comparison of the results revealed that XGBoost Regressor was better than LightGBM Regressor and it outperforms LightGBM Regressor in all the performance measurements. The XGBoost model had a Mean Absolute Error of 7.17 minutes which means that the model had an error margin of seven minutes on average with the actual delay duration. It also was able to control larger prediction errors with a better root mean squared error of 17.26 minutes. Moreover, the model attained an R 2 Score of 0.7693, which implies that the model was able to explain about 76.9 percent of the difference in delay time.

Regression Validation Results

The LightGBM Regressor was tested on the held out 10% validation set to test its generalization performance. The model had a mean absolute error (MAE) of 22.97 minutes, which means that predictions were in general 23 minutes off actual delay time. The root mean squared error (RMSE) was 55.94 minutes which indicates the effects of bigger errors in the distribution. The coefficient of determination (R 2) was 0.6968 indicating that out of the log transformed delay minutes, the model explained about 70 percent of the variance. Although these findings support the idea that LightGBM was able to capture meaningful patterns within the data, its error rates were higher than those of XGBoost, supporting the fact that XGBoost is the more robust regression model to predict flight delay durations.

Metric	Value
MAE (Minutes)	22.97
RMSE (Minutes)	55.94
R^2 Score	0.6968

Table11: Regression Validation Results

Regression — Error Analysis

To measure further the performance of the regression, the difference between the actual and predicted delay minutes on the validation set was measured as the residuals. The distribution of residuals is shown in Figure8

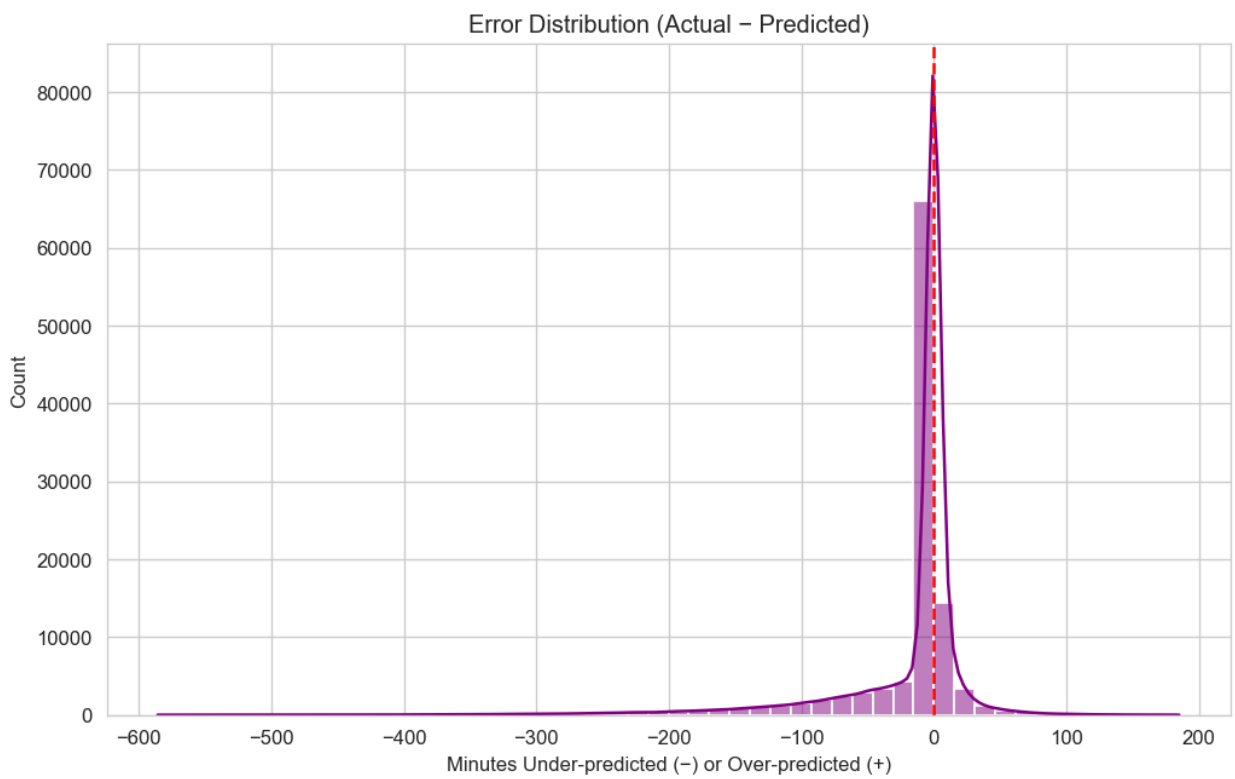

Fig8: Error Distribution (Actual − Predicted)

The histogram shows that there is a concentration of most of the errors around the zero point with a sharp peak showing that most of the predictions closely coincide with the actual delay times. Distribution is roughly centered with a mean residual of 0.00 minutes, which ascertains that the model does not systematically over predict or under predict delays. The great length of the tails in either direction, however, emphasize the infrequent huge deviations, showing how much variability there is in flight business, and how hard it is to model extreme delayed times.

This analysis of residual values (MAE = 22.97 minutes, RMSE = 55.94 minutes, R 2 = 0.6968) is a visual confirmation that the LightGBM Regressor can effectively represent the majority of delay patterns, but still struggles with extreme cases.

Regression — Actual vs. Predicted

In order to visualize regression performance, the predicted delay minutes versus actual delay minutes of the held-out test set were plotted and zoomed to the 99 th percentile in order to minimize the impact of extreme outliersFigure 9shows the scatter of predictions compared to the diagonal perfect prediction line.

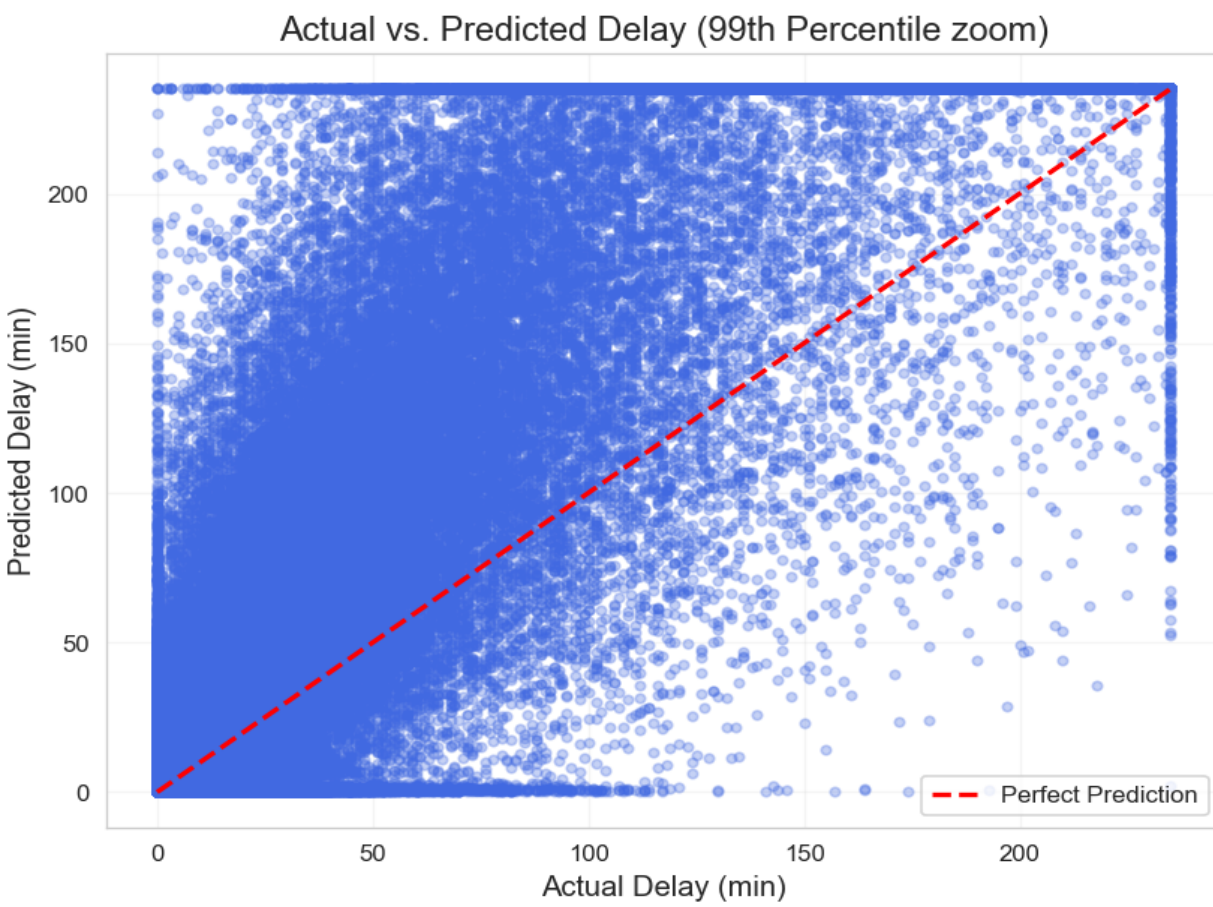

Fig9: Actual vs. Predicted Delay

Most of the points are closely clustered around the diagonal, which proves that the central tendency of delay durations is well represented by the model. Deviations off the line are an indication of errors in prediction, some under prediction and over prediction are evident, especially when the delay is longer. The visualization highlights the accuracy of the model at the 99th percentile of flight delays that are typical, but also that extreme cases are harder to predict.

This analysis is complementary to the residual distribution and validation metrics, which supports the idea that the LightGBM Regressor can predict the majority of flights with good accuracy, whereas XGBoost showed a higher overall accuracy and lower error rates.

Regression — Feature Importance

To interpret the regression model, the LightGBM Regressor provided the feature importance scores. Figure 10 indicates the strongest predictors of minutes of flight delay.

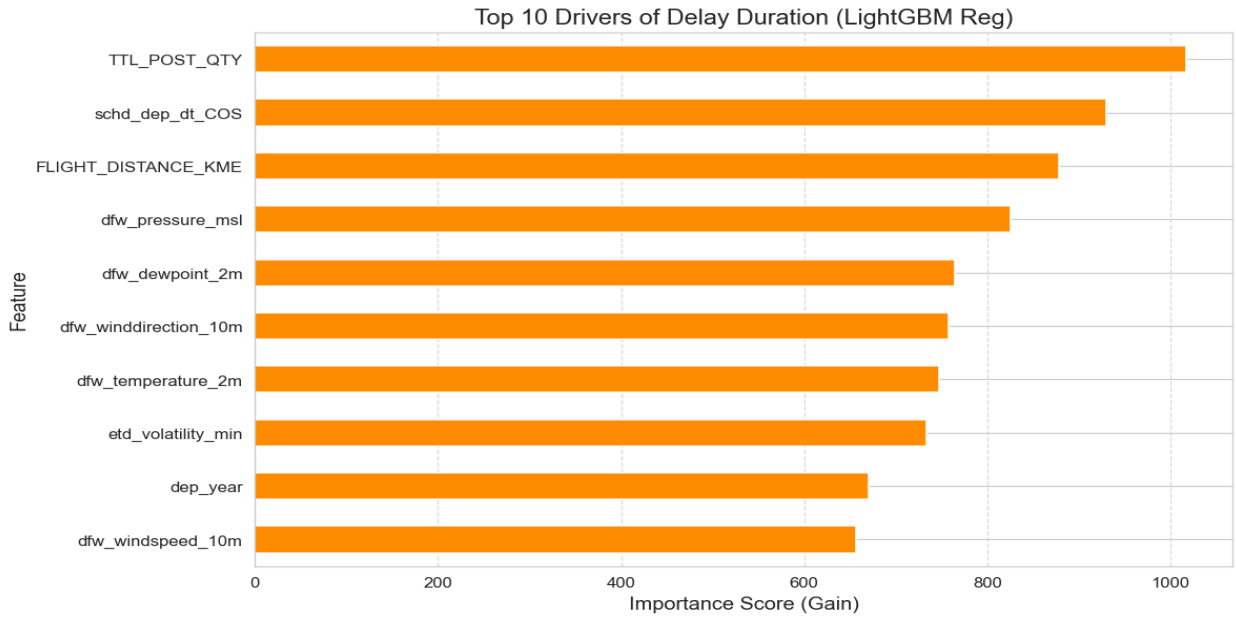

Fig10: Top 10 Drivers of Delay Duration(LightGBM Reg)

As it can be seen, operational signals like TTL_POST_QTY and etd_volatility_min were some of the strongest drivers indicating the significance of post departure updates and volatility in estimated times. These temporal encodings such as schd_dep_dt_COS were among the highest ranking encodings, since they absorbed cyclical patterns in departure times. The variables of route and distance (FLIGHT_DISTANCE_KME, dep_year) played a significant role and weather variables like dfw_pressure-msl, dfw-dewpoint-2m, dfw-temperature-2m, and dfw-windspeed-10m provided essential background to the operational disruption.

Such ranking ascertains that the model incorporates an equal balance of operational, temporal, route, and weather characteristics to forecast delay periods. The analysis is more useful in terms of interpretability and actionable insights on factors that play the most significant role in flight delays by quantifying the importance of features.

Hyperparameter Tuning and Final Model Optimization

Hyperparameter tuning was conducted to enhance the performance of the 23odeling with the help of RandomizedSearchCV that is an efficient way to search a subset of parameter combinations than exhaustive grid search.

XGBoost Classifier Tuning :

In the classification task, the XGBoost classifier was used with RandomizedSearchCV to find the best combination of parameters with many candidate values. The tuning experiment investigated the changes in the number of trees, maximum tree depth, learning rate, minimum child weight, gamma, subsampling ratio and column sampling ratio. The 3-fold cross-validation was performed on 15 random combinations of parameters, and AUC-ROC was adopted as a scoring metric. This method facilitated effective search of the hyperparameter space as well as lowering the cost of computation than exhaustive grid search.

The tuning process selected a set of the best XGBoost classification parameters as follows: 300 estimators, maximum depth of 4, learning rate of 0.01, minimum child weight of 10, subsample ratio of 0.7 and column sampling ratio of 0.8. These tune parameters enhanced the balance of the model between predictive and generalization.

Parameter	Value
n_estimators	300
max_depth	4
learning_rate	0.01
min_child_weight	10
gamma	0
subsample	0.7
colsample_bytree	0.8

Table12: Tuned Hyperparameters of XGBoost

In the case of the regression task, the regression model that was tested with RandomizedSearchCV was the LightGBM Regressor. The tuning procedure considered the number of trees, number of leaves, maximum depth, learning rate, regularization parameters, and minimum number of child samples. The 15 combinations of parameters were tested with the help of 3-fold cross-validation and negative mean absolute error as the evaluation measure. The optimum regression setup identified in this process was 700 estimators, 31 leaves, maximum depth of 6, learning rate of 0.01, minimum child samples of 100, and regularization parameter lambda of 0.1.

Parameter	Value
n_estimators	700
num_leaves	31

max_depth	6
learning_rate	0.01
reg_alpha	0
reg_lambda	0.1
min_child_samples	100

Table13: Tuned Hyperparameters of LightGBM

Upon hyperparameter optimization, the optimized XGBoost classifier was retrained on the training data with the classification threshold being optimized to enhance the F1-score. The default probability threshold of 0.50 was not used but a number of threshold values between 0.30 and 0.75 were tested and the one that gave the best F1-score was chosen. The best trade off was identified to be 0.45 giving the best compromise between precision and recall in delayed flight detection.

Model Tuning Comparison – Before vs After Tuning

Classification Model (XGBoost)

Model	Accuracy	Recall (Del.)	F1 (Del.)	AUC-ROC	Recommendation
XGBoost (Tuned)	89.00%	77.00%	0.8108	0.9194	✓ Selected
XGBoost (Default)	88.10%	81.59%	0.8022	0.9171	Strong

Regression Model (LightGBM)

Model	MAE (min)	RMSE (min)	R^2	Recommendation
LightGBM (Tuned)	21.39	56.80	0.6257	✓ Selected
LightGBM (Default)	21.52	57.09	0.6220	Strong

The tuning results show that hyperparameter optimization led to modest improvements in model performance. For the classification task, the tuned XGBoost model achieved a higher AUC-ROC and F1-score compared to the default model, although recall slightly decreased due to threshold optimization.

For the regression task, the tuned LightGBM model showed slight reductions in MAE and RMSE along with an improved R^2 score, indicating better predictive accuracy. Overall, tuning enhanced model performance while maintaining strong generalization.

FINAL TUNED MODEL RESULTS

The last tuned XGBoost classifier had a AUC-ROC score of 0.8588 on the testing set, and a total accuracy of 84%. In the case of delayed flights, the model was found to be 92 percent precise, 67 percent recall and F1-score of 0.77, which is a good result in terms of detecting delay events and reducing false alarms.

Metric	Value
Accuracy	0.84
AUC-ROC	0.8588
F1-Score (Delayed)	0.77

Table14: Final Tuned Model Results For Classification

In the regression task, the tuned LightGBM Regressor had a Mean Absolute Error of 7.80 minutes, a Root Mean Squared Error of 18.06 minutes, and an R2 Score of 0.7585 on the test data. These findings suggest that the final regression model could predict the delay duration with a relatively small error as well as it was able to explain a significant portion of the delay duration variation (approximately 75.9%).

Metric	Value
MAE	7.80 minutes
RMSE	18.06 minutes
R^2 Score	0.7585

Table15: Final Tuned Model Results For Regression

SHAP:

The tuned XGBoost classifier was applied to SHAP (Shapley Additive explanations), to enhance interpretability and to give insight on how predictions were achieved. It was found that the most significant drivers of delay detection were the operational signals like MINS_TO_SCHD_DEP_QTY, TTL_POST_QTY, and etd_volatility_min, as they indicated the significance of pre-departure updates and volatility of the estimated departure time. Temporal characteristics such as dep_year, dep_hour and dep-month-cos also contributed significantly, which reflected cyclical and seasonal trends in flight operations. Moreover, weather factors such as dfw_pressure_msl, dfw_precipitation, dfw_relativehumidity 2m, and dfw dewpoint 2m also played a significant role, indicating the influence of the environmental factors on delay risk. Additional route-level characteristics like FLIGHT_DISTANCE_KME and route_freq were also included as they provided additional information based on the variation in congestion and operational complexity among flight paths.

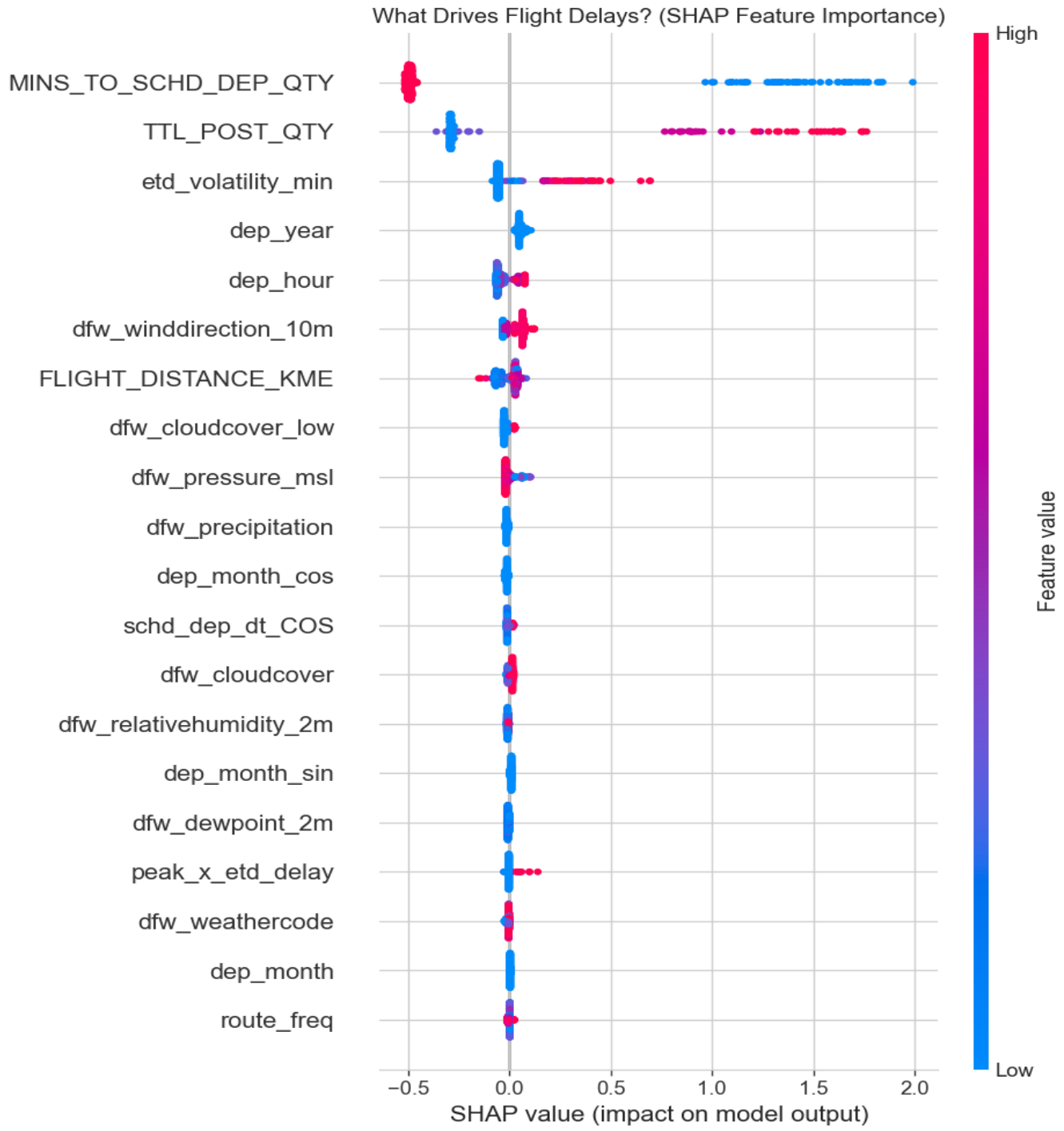

Fig11: What Drives Flight Delays? (SHAP Analysis)

Conclusion

In this paper, a two-level machine learning model was developed and tested with a large amount of flight operation data supplemented by past weather observations to predict flight arrival delays. The results give clear answers to the questions raised in the research.

Research Question 1: **Can we forecast whether an aircraft will be delayed before takeoff?**

Yes. The XGBoost classifier configured to run on tuned mode had an overall accuracy of 84% and AUC ROC of 0.8588 on the held out test set with a precision of 0.92 on delayed flights and a recall of 0.96 on time flights. These findings show that the system is able to detect delays effectively and in time and thus it is viable to implement the system in airline monitoring systems.

Research Question 2: **Can we forecast the length of a delayed flight?**

To a strong degree. The LightGBM regressor when tuned obtained a R2 of 0.7585, mean absolute error of 7.80 minutes, and RMSE of 18.06 minutes on the test set. Although the uncertainty is still there because of the complexity and cascading effect of delays in the aviation network, the model accounts more than 75% of the variation in delay duration. Notably, the regressor is biased towards a slight overestimation of delay length, which is a less risky bias in operation than underestimation.

Combined, these findings ratify that the two stage model is effective and interpretable, and provides actionable information on delay detection and duration forecasting that can aid airlines in streamlining schedules, resource distribution, and communication with passengers.